RaktSetu — Team Learning Roadmap

> **A step-by-step guide for 5 beginners to recreate this entire project from scratch**

Project Overview

RaktSetu is a multi-page blood donation platform with:

Layer	Technologies
Build Tool	Vite
UI Framework	React 19
Routing	React Router DOM v7
Icons	Lucide React
Animations	Framer Motion
Styling	Vanilla CSS
State Management	React Context API + localStorage
Deployment	Vercel

Full File Map (What You're Building)

```
raktsetu-platform/
├── index.html                    ← Entry HTML file
├── package.json                  ← Project config + dependencies
├── vite.config.js                ← Build tool configuration
├── src/
│   ├── main.jsx                  ← React entry point
│   ├── App.jsx                   ← Route definitions
│   ├── index.css                 ← Global styles
│   ├── App.css                   ← App-level styles
│   ├── components/               ← Reusable UI pieces
│   │   ├── Navbar.jsx + .css     ← Navigation with dropdowns
│   │   ├── Footer.jsx + .css     ← Site footer
│   │   ├── Layout.jsx + .css     ← Page wrapper (Navbar + content + Footer)
│   │   ├── Button.jsx + .css     ← Reusable button
│   │   ├── FeatureCard.jsx + .css← Feature display card
│   │   ├── AnimatedSection.jsx   ← Scroll-based animations
│   │   ├── CountUp.jsx           ← Animated number counter
│   │   ├── ErrorBoundary.jsx     ← Error catching wrapper
│   │   └── ScrollToTop.jsx       ← Auto-scroll on navigation
│   ├── pages/                    ← Full page content
│   │   ├── Home.jsx + .css       ← Landing page (largest: 477 lines)
│   │   ├── About.jsx + .css      ← Mission & team info
│   │   ├── Features.jsx + .css   ← Feature showcase
```

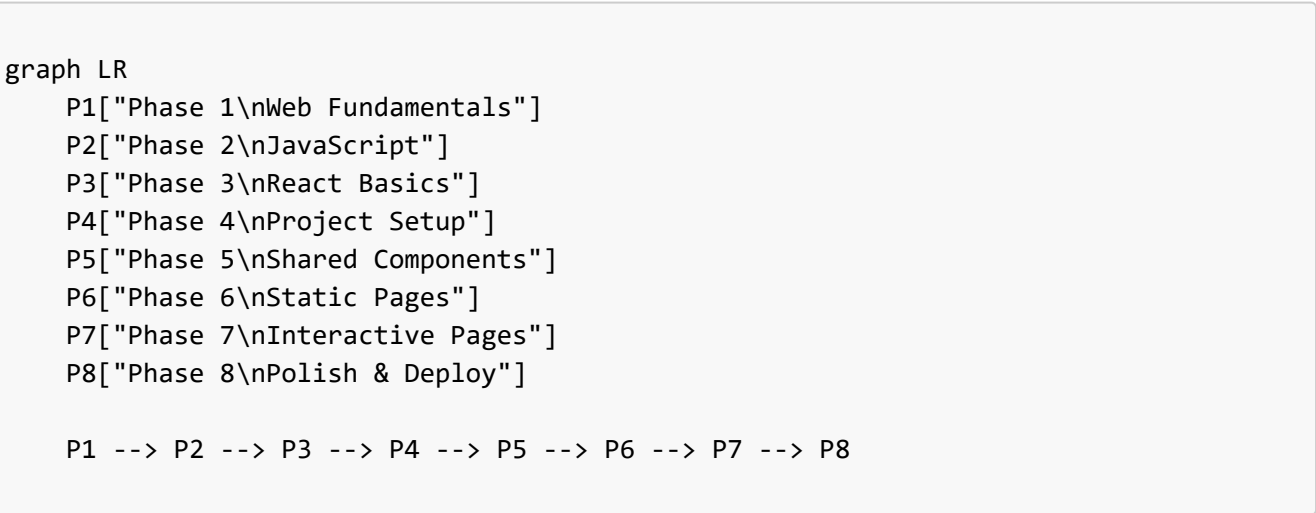
```
│   │   ├── HowItWorks.jsx + .css ← Step-by-step donor guide
│   │   ├── ForBloodBanks.jsx+.css← Blood bank partner page
│   │   ├── Contact.jsx + .css    ← Contact form with validation
│   │   ├── BloodAvailability.jsx ← Search blood banks (with filters)
│   │   ├── DonorRegistration.jsx ← Multi-field donor signup form
│   │   ├── EmergencyRequest.jsx  ← Urgent blood request form
│   │   ├── Login.jsx + .css      ← Auth with tabs (Donor/Admin)
│   │   ├── PrivacyPolicy.jsx     ← Legal: Privacy
│   │   ├── Terms.jsx             ← Legal: Terms of Service
│   │   └── NotFound.jsx + .css   ← 404 error page
│   ├── context/
│   │   └── AuthContext.jsx       ← Authentication state management
│   └── data/
│       ├── bloodBanks.js         ← Mock blood bank database
│       └── storage.js            ← localStorage helper functions
```

👥 Suggested Team Roles

Member	Role	Primary Focus
Member 1	**Project Lead + Layout**	HTML structure, Navbar, Footer, Layout, routing
Member 2	**Styling Lead**	All CSS files, responsive design, animations
Member 3	**Forms & Validation**	Contact, DonorRegistration, EmergencyRequest, Login
Member 4	**Data & Logic**	Mock databases, search/filter logic, Context API, localStorage
Member 5	**Content Pages**	Home, About, Features, HowItWorks, ForBloodBanks, Legal pages

> [!TIP] Everyone should learn the fundamentals together (Phases 1–3). Specialization starts at Phase 4.

🗺️ The 8 Phases



Phase 1 — Web Fundamentals (Everyone) 📖

> **Duration:** ~2 weeks | **Goal:** Build a static webpage from scratch

What to Learn

1.1 HTML — The Skeleton

Topic	Why You Need It	Where It's Used in RaktSetu
Tags: `<div>`, `<h1>`–`<h6>`, `<p>`, `<span>`	Structure all content	Every single JSX file
`<a>` (links), `<img>`	Navigation and media	Footer, Navbar
`<form>`, `<input>`, `<textarea>`, `<select>`, `<button>`, `<label>`	All forms	Contact, DonorRegistration, EmergencyRequest, Login
`<ul>`, `<li>`	Lists and menus	Navbar dropdown menus, Footer link columns
`<header>`, `<nav>`, `<main>`, `<footer>`, `<section>`	Semantic structure	Layout, Navbar, Footer
HTML attributes: `id`, `class`, `placeholder`, `required`, `type`, `aria-label`	Accessibility & form behavior	All forms and interactive elements

1.2 CSS — The Skin

Topic	Why You Need It	Where It's Used
Selectors (`.class`, `#id`, element)	Target elements for styling	Every `.css` file
Box Model (`margin`, `padding`, `border`)	Spacing and sizing	All components
Flexbox (`display: flex`, `justify-content`, `align-items`, `gap`)	Horizontal/vertical layouts	Navbar, Footer grid, form layouts
CSS Grid (`display: grid`, `grid-template-columns`)	2D multi-column layouts	Footer columns, feature card grids, contact page
Colors, gradients (`linear-gradient`)	Visual design	Hero sections, buttons, backgrounds
Typography (`font-family`, `font-size`, `font-weight`, `line-height`)	Text readability	Global styles in `index.css`
Pseudo-classes (`:hover`, `:focus`, `:active`)	Interactive states	Buttons, links, inputs
`@media` queries	Responsive design for mobile/tablet	Every CSS file's bottom section
CSS Variables (`--var-name`)	Design tokens / theme colors	`index.css` (global design system)

Topic	Why You Need It	Where It's Used
`position` (`relative`, `absolute`, `fixed`, `sticky`)	Dropdowns, sticky navbar	Navbar dropdown, ticker bar
`transition` and `animation` / `@keyframes`	Smooth hover effects, spinners	Button hover, page loader spinner
`overflow`, `z-index`	Dropdown menus, overlays	Navbar mobile menu

☑ Phase 1 Checkpoint

Build this without any framework:

- ☐ A single `index.html` page with a header, navigation bar, a hero section, 3 feature cards in a grid, and a footer
- ☐ Style it with a separate `style.css` file
- ☐ Make it responsive (looks good on phone + desktop)
- ☐ Navigation links should have hover effects

> [!IMPORTANT] **Do NOT skip this phase.** If you can't build a multi-section HTML/CSS page from memory, React will feel impossible.

📚 Recommended Resources

- MDN Web Docs — HTML
- MDN Web Docs — CSS
- Flexbox Froggy (game to learn Flexbox)
- Grid Garden (game to learn CSS Grid)
- Kevin Powell's YouTube (best CSS tutorials)

Phase 2 — JavaScript Essentials (Everyone) ▨

> **Duration:** ~2–3 weeks | **Goal:** Make web pages interactive

What to Learn

2.1 Core JavaScript

Topic	Why You Need It	Where It's Used
Variables (`let`, `const`)	Store and track data	Every component's state
Data types (string, number, boolean, array, object)	Structure information	Form data, blood bank records
Functions (regular + arrow `() => {}`)	Reusable logic blocks	Every React component is a function
Template literals (`` `Hello ${name}` ``)	Dynamic strings	JSX expressions

Topic	Why You Need It	Where It's Used
Conditionals (`if`/`else`, ternary `? :`)	Show/hide things conditionally	Error messages, loading states, active tabs
Loops (`for`, `.map()`, `.filter()`, `.find()`)	Render lists of data	Nav items, blood bank results, feature cards
Objects: destructuring `{ name, age } = person`	Extract values cleanly	Props, form data, event handlers
Spread operator (`...obj`)	Copy/merge objects	`setFormData({ ...formData, [name]: value })`
Array methods: `.map()`, `.filter()`, `.find()`, `.length`	Data transformation	Blood bank filtering, form validation

2.2 DOM & Events (Bridges HTML ↔ JS)

Topic	Why You Need It	Where It's Used
`document.querySelector()`, `getElementById()`	Select HTML elements	Understanding how React replaces manual DOM
Event listeners (`click`, `submit`, `change`)	Handle user actions	Every button, form, input
`event.preventDefault()`	Stop form from refreshing page	Every form `handleSubmit` function
`event.target.name`, `event.target.value`	Read input values	`handleChange` functions in all forms

2.3 Modern JavaScript (ES6+)

Topic	Why You Need It	Where It's Used
`import` / `export`	Split code into files	Every file imports/exports
`async` / `await` + `Promise`	Handle delays (API calls)	Login, Contact form (simulated delays)
Optional chaining `?.`	Safe property access	`user?.role === 'admin'` in AuthContext
`JSON.parse()` / `JSON.stringify()`	Read/write to localStorage	AuthContext, storage.js

2.4 Browser APIs

Topic	Why You Need It	Where It's Used
`localStorage.getItem()` / `.setItem()` / `.removeItem()`	Persist data between page loads	AuthContext (user sessions), storage.js (donor data)

Topic	Why You Need It	Where It's Used
`setTimeout()`	Simulate delays, auto-dismiss messages	Form submission feedback
`console.log()` / `console.error()`	Debugging	ErrorBoundary, AuthContext

☑ Phase 2 Checkpoint

Build this in vanilla JS (no framework):

- ☐ A to-do list app: Add items, mark complete, delete items, stored in `localStorage`
- ☐ A search filter: Given an array of blood bank objects, display them as cards and filter by typing a name
- ☐ A form with validation: Name, email, phone — show error messages for invalid inputs

📚 Recommended Resources

- JavaScript.info (best structured JS course)
- MDN JavaScript Guide
- Fireship — JavaScript in 100 seconds

Phase 3 — React Fundamentals (Everyone) ⚛

> **Duration:** ~3 weeks | **Goal:** Build simple React components

What to Learn

3.1 React Core Concepts

Concept	What It Is	Where It's Used
JSX	HTML-like syntax inside JavaScript	Every `.jsx` file — it's how you write UI in React
Components	Reusable UI building blocks (functions that return JSX)	`Button`, `FeatureCard`, `Navbar`, every page
Props	Passing data from parent → child	`<Button variant="primary">`, `<FeatureCard title="..." />`
`useState`	Store data that changes over time	Form inputs, toggles, loading states, errors
`useEffect`	Run code on component mount/update	Load user from localStorage, add event listeners
Conditional Rendering	Show different UI based on state	`{error && <span>{error}</span>}`, `{loading ? 'Loading...' : 'Submit'}`

Concept	What It Is	Where It's Used
List Rendering	`.map()` over arrays to create multiple elements	Nav items, blood bank results, feature cards
Event Handling	`onClick`, `onChange`, `onSubmit`	Every interactive element

3.2 React Patterns Used in This Project

Pattern	Example in RaktSetu	File
Controlled inputs	`value={formData.name} onChange={handleChange}`	Contact.jsx, DonorRegistration.jsx
Form state object	`const [formData, setFormData] = useState({...})`	All form pages
Computed/derived state with `useMemo`	`const filteredResults = useMemo(...)`	BloodAvailability.jsx
Loading / submitting states	`const [loading, setLoading] = useState(false)`	Login.jsx, Contact.jsx
Success/error feedback	Show success banner after form submit, then auto-hide	Contact, DonorRegistration

3.3 Tools You'll Need Installed

Tool	Purpose	Install Command
Node.js (v18+)	Run JavaScript outside browser, package manager	Download from nodejs.org
npm	Install packages (comes with Node)	Comes with Node.js
VS Code	Code editor	Download from code.visualstudio.com
Git	Version control	Download from git-scm.com

☑ Phase 3 Checkpoint

- ☐ Create a **counter app** with `useState` (increment/decrement buttons)
- ☐ Create a **card component** that takes `title`, `description`, `icon` as props
- ☐ Create a **form component** that handles input changes with a single `handleChange` function
- ☐ Render a **list of items** from an array using `.map()`

🔗 Recommended Resources

- React Official Tutorial (the new React docs are excellent)
- React.dev — Thinking in React

- Scrimba React Course (Free)

Phase 4 — Project Setup (Member 1 leads, Everyone follows) 🏗️

> **Duration:** ~2–3 days | **Goal:** Empty project running in browser

Step-by-Step

4.1 Create the Vite + React Project

Skills needed: Command line basics, npm

```
# Create the project
npx -y create-vite@latest raktsetu-platform -- --template react

# Enter the project
cd raktsetu-platform

# Install dependencies
npm install

# Start the dev server
npm run dev
```

What to understand:

- `package.json` — lists your project name, scripts (`dev`, `build`), and dependencies
- `vite.config.js` — configures the build tool (you'll add optimizations later)
- `src/main.jsx` — the entry point: renders `<App />` into the HTML page
- `index.html` — the only HTML file; React injects everything into `<div id="root">`

4.2 Install Dependencies

```
npm install react-router-dom lucide-react framer-motion
```

Package	Purpose	Can be added later?
`react-router-dom`	Multi-page navigation without page reload	❌ Needed from Phase 5
`lucide-react`	Beautiful icon library (used everywhere)	☑ Yes, can use text first
`framer-motion`	Scroll animations, page transitions	☑ Yes, added in Phase 8

4.3 Set Up Folder Structure

Create the empty folder structure:

```
src/
├── components/
├── pages/
├── context/
├── data/
└── assets/
```

4.4 Set Up Global Styles

Create `src/index.css` with your design tokens:

- CSS variables for colors (primary red `#8B0000`, backgrounds, text colors)
- Base font family (e.g., Google Fonts: Inter or Outfit)
- CSS reset (normalize margins, box-sizing)
- Global utility classes (`.container`, `.section`, `.glass-panel`)

4.5 Set Up Git

```
git init
git add .
git commit -m "Initial project setup with Vite + React"
```

☑ Phase 4 Checkpoint

- ☐ Running `npm run dev` opens a page in the browser
- ☐ You see your custom styles applied
- ☐ All team members can clone the repo and run it locally
- ☐ Folder structure is created

Phase 5 — Shared Components (Member 1 + Member 2) 🧩

> **Duration:** ~1–2 weeks | **Goal:** Build the reusable building blocks

Build these in order — each one builds on the previous:

5.1 `Button.jsx` + `Button.css`

Skills needed: Props, CSS classes, conditional styling

What it does: A reusable button with variants (`primary`, `outline`, `ghost`)

```
Files: src/components/Button.jsx (~20 lines), Button.css (~60 lines)
```

What to learn:

- Accepting props: `({ children, variant, className, ...rest })`
- Spreading `...rest` to pass unknown props (like `onClick`, `type`)
- CSS class composition: combining `btn btn-primary my-custom-class`

Test it: Render 3 buttons with different variants on a blank page. Verify hover effects work.

5.2 `ScrollToTop.jsx`

Skills needed: `useEffect`, `useLocation` from React Router

What it does: Scrolls to top of page whenever the URL changes (so users don't land mid-page)

```
Files: src/components/ScrollToTop.jsx (~12 lines)
```

What to learn:

- `useLocation()` hook — gives you the current URL path
- `useEffect` dependency array — run code when URL changes
- `window.scrollTo(0, 0)` — browser scroll API

Test it: Navigate between two pages and verify you always start at the top.

5.3 `Layout.jsx` + `Layout.css`

Skills needed: React Router `<Outlet />`, component composition

What it does: Wraps every page with Navbar on top, Footer on bottom, and page content in between

```
Files: src/components/Layout.jsx (~20 lines), Layout.css (~5 lines)
```

What to learn:

- `<Outlet />` — React Router's way of saying "render the child route's page here"
- Component composition — nesting components inside each other

Test it: Wrap a dummy page inside Layout. Verify Navbar appears on top, Footer on bottom.

5.4 `Navbar.jsx` + `Navbar.css` *Most Complex Component*

Skills needed: `useState`, `useRef`, `useEffect`, arrays, `.map()`, conditional rendering

What it does: Top header with logo + search bar, navigation bar with dropdown menus, scrolling ticker

```
Files: src/components/Navbar.jsx (~166 lines), Navbar.css (~200 lines)
```

Break it into micro-steps:

Step	What to Build	New Concept
5.4a	Static header with logo text and a search input	Flexbox layout, brand styling
5.4b	Horizontal navigation links (no dropdowns)	`<NavLink>` from React Router, active link styling
5.4c	Navigation data array + render with `.map()`	Array of objects, `.map()` rendering
5.4d	Dropdown menus on hover/click	`useState` for `activeDropdown`, conditional class `show`
5.4e	Click-outside-to-close dropdowns	`useRef`, `useEffect`, `document.addEventListener`
5.4f	Mobile hamburger menu toggle	`useState` for `isOpen`, responsive CSS
5.4g	Ticker/marquee bar at bottom	CSS animation or static text bar

Test each step independently before moving to the next.

5.5 `Footer.jsx` + `Footer.css`

Skills needed: CSS Grid, `<Link>` from React Router, icon components

What it does: 4-column footer with brand info, link lists, contact info, social icons

```
Files: src/components/Footer.jsx (~88 lines), Footer.css (~100 lines)
```

What to learn:

- CSS Grid for multi-column layout
- `<Link to="/path">` vs `<a href="">` — internal vs external links
- `target="_blank" rel="noopener noreferrer"` for external links (security)

Test it: Verify all 4 columns render, links work, and it collapses to single column on mobile.

5.6 `FeatureCard.jsx` + `FeatureCard.css`

Skills needed: Props, icon components

What it does: A card with an icon, title, and description

```
Files: src/components/FeatureCard.jsx (~15 lines), FeatureCard.css (~30 lines)
```

Test it: Render 6 cards in a grid on a blank page.

5.7 `ErrorBoundary.jsx`

Skills needed: React class component (the only one in the project), `componentDidCatch`

What it does: If any child component crashes, shows a friendly error message instead of a blank screen

```
Files: src/components/ErrorBoundary.jsx (~70 lines)
```

What to learn:

- This is the **only class component** — Error Boundaries can't be function components (React limitation)
- `static getDerivedStateFromError()` and `componentDidCatch()` lifecycle methods

Test it: Deliberately throw an error in a child component and verify the error screen shows.

5.8 Set Up Routing in `App.jsx`

Skills needed: React Router (`BrowserRouter`, `Routes`, `Route`), `lazy()`, `Suspense`

What it does: Maps URL paths to page components

```
Files: src/App.jsx (~74 lines)
```

Build in stages:

Step	What to Add
5.8a	Basic routing: `/` → Home, `/about` → About (just placeholder `<h1>` pages)
5.8b	Nested route inside `<Layout />` so all pages get Navbar + Footer
5.8c	`*` catch-all route for 404 NotFound page
5.8d	Lazy loading with `React.lazy()` + `<Suspense>` + loading spinner

Test it: Click navigation links. Verify URL changes, correct page loads, and Navbar/Footer persist.

☑ Phase 5 Checkpoint

- ☐ Navbar with working dropdowns and mobile menu

- [ ] Footer with all links
- [ ] Navigation between placeholder pages works
- [ ] 404 page shows for invalid URLs
- [ ] Page always scrolls to top on navigation

Phase 6 — Static Content Pages (Member 5 leads)

> **Duration:** ~1–2 weeks | **Goal:** Fill in all informational pages

These pages are essentially "styled HTML" — no complex logic, mostly JSX + CSS.

6.1 `NotFound.jsx` + `NotFound.css` (Easiest Page — Start Here)

Skills needed: Basic JSX, `<Link>`, CSS centering

What it does: A 404 page with "Page Not Found" message and a "Go Home" button

```
Complexity: ★ (~30 lines)
```

6.2 `About.jsx` + `About.css`

Skills needed: Sections, semantic HTML, CSS styling

What it does: Mission statement, team/values section, vision cards

```
Complexity: ★★ (~90 lines)
```

6.3 `Features.jsx` + `Features.css`

Skills needed: Reusing `<FeatureCard>` component, arrays + `.map()`

What it does: Grid of feature cards showcasing platform capabilities

```
Complexity: ★★ (~100 lines)
```

Key pattern: Define a `features` array of objects, then `.map()` over them to render `<FeatureCard>` components.

6.4 `HowItWorks.jsx` + `HowItWorks.css`

Skills needed: Numbered steps, timeline/stepper layout

What it does: Step-by-step guide showing the donation process with numbered steps and icons

```
Complexity: ★★ (~130 lines)
```

6.5 `ForBloodBanks.jsx` + `ForBloodBanks.css`

Skills needed: Sections with alternating layouts, CTA (call-to-action) buttons

What it does: Information page targeted at blood banks wanting to partner

```
Complexity: ★★ (~150 lines)
```

6.6 `PrivacyPolicy.jsx` + `Terms.jsx` + `Legal.css`

Skills needed: Just long-form text content in JSX

What it does: Standard legal pages

```
Complexity: ★ (~90 lines each)
```

☑ Phase 6 Checkpoint

- [ ] All info pages render with proper styling
- [ ] Navigation to every page works
- [ ] Pages are responsive on mobile
- [ ] Each page has a hero/banner section at the top

Phase 7 — Interactive Pages (Member 3 + Member 4) 🔧

> **Duration:** ~3–4 weeks | **Goal:** Build forms, search, auth, and data logic

This is the most challenging phase. Build in this exact order.

7.1 Mock Data Layer (Member 4)

Build the data foundation before any UI.

7.1a `data/bloodBanks.js`

Skills needed: Arrays of objects, `export`

What it does: A hardcoded array of ~50 blood bank objects, each with `name`, `city`, `state`, `phone`, `bloodGroups` (with availability counts), `hours`, `type`

```
Complexity: ★★ (lots of data, simple structure — ~400 lines)
```

What to learn:

- Structured data modeling — designing what fields a blood bank needs
- Named exports: `export const bloodBanks = [...]`
- Helper functions: `getStats()` returning totals, `getBloodBanks()` with optional filters

---

7.1b `data/storage.js`

Skills needed: `localStorage`, `JSON.parse/stringify`, helper functions

What it does: Utility functions for localStorage operations

```
Complexity: ★★ (~80 lines)
```

Functions to build:

- `registerDonor(donorData)` — save donor to localStorage array
- `getDonors()` — get all saved donors
- `getDonorCount()` — return count
- `submitEmergencyRequest(requestData)` — save emergency request
- `getEmergencyRequests()` — get all requests

Test it: Call each function from browser console. Verify data persists after page refresh.

7.2 `Contact.jsx` + `Contact.css` (Simplest Form)

Skills needed: `useState`, controlled inputs, form validation, async simulation

What it does: Contact form (name, email, subject, message) with validation and submission feedback

```
Complexity: ★★★ (~173 lines)
```

Build step by step:

Step	What to Build	Concept

Step	What to Build	Concept
7.2a	Form layout (4 fields) with CSS	HTML form, CSS grid
7.2b	Controlled inputs with `useState`	`formData` state object, `handleChange`
7.2c	Validation on submit	`validate()` function, `errors` state
7.2d	Inline error messages	Conditional rendering `{errors.name && <span>...}`
7.2e	Simulated submit with loading state	`async/await`, `setTimeout`, `submitState`
7.2f	Success message with auto-dismiss	`setTimeout(() => setSubmitState('idle'), 4000)`

Test it: Submit with empty fields → errors show. Fill all fields → success message appears and disappears.

7.3 `DonorRegistration.jsx` + `DonorRegistration.css`

Skills needed: Everything from Contact + more complex validation + `storage.js`

What it does: Multi-field donor registration form saving to localStorage

```
Complexity: ★★★★ (~265 lines)
```

Additional fields: Full name, email, phone, date of birth, blood group (dropdown), city, state, password, terms checkbox

New concepts beyond Contact.jsx:

- `<select>` dropdown for blood group
- Checkbox for terms agreement
- More complex validation rules (phone format, age >= 18, password length)
- Calling `registerDonor()` from `storage.js` to persist data
- Success screen with registration ID

Test it: Register a donor → check `localStorage` in browser DevTools → verify the data is saved correctly.

7.4 `EmergencyRequest.jsx` + `EmergencyRequest.css`

Skills needed: Same patterns as DonorRegistration

What it does: Emergency blood request form with urgency level, hospital info, units needed

```
Complexity: ★★★★ (~212 lines)
```

Test it: Submit request → verify it persists in localStorage.

7.5 `BloodAvailability.jsx` + `BloodAvailability.css`

Skills needed: `useState`, `useMemo`, filtering arrays, computed values

What it does: Search and filter blood banks by state, city, blood group, and component type

```
Complexity: ★★★★★ (~185 lines — most complex logic)
```

Build step by step:

Step	What to Build	Concept
7.5a	Import and display all blood banks as cards	`.map()` rendering, card layout
7.5b	Add state filter dropdown	`useState` for `filters`, filter on `state`
7.5c	Add blood group filter pills	Array of blood groups, toggle active filter
7.5d	Add text search input	Filter by `name` text match
7.5e	Combine all filters with `useMemo`	`useMemo(() => bloodBanks.filter(...), [dependencies])`
7.5f	Result count + clear filters button	Derived state, reset handler
7.5g	"No results found" empty state	Conditional rendering

What to learn about `useMemo`:

- It **memoizes** (caches) the result of an expensive computation
- Only re-runs when its dependency values change
- Without it, filtering 50 blood banks on every render slows things down

Test it: Verify each filter works independently and in combination. Clear filters resets all.

7.6 `context/AuthContext.jsx` + `Login.jsx` (Member 4 leads)

Skills needed: React Context API, `createContext`, `useContext`, `Provider`

7.6a `AuthContext.jsx`

What it does: Provides app-wide user authentication state

```
Complexity: ★★★★ (~45 lines)
```

What to learn:

- `createContext()` — creates a "global state container"

- `AuthProvider` wraps the app and provides `{ user, login, logout, isAdmin, loading }`
- `useAuth()` custom hook — any component can access auth state
- Persisting user session in `localStorage`

7.6b `Login.jsx` + `Login.css`

What it does: Login form with Donor/Admin tab switch, mock authentication

```
Complexity: ★★★★ (~136 lines)
```

Build step by step:

Step	What to Build	Concept
7.6b-1	Login form UI (email + password)	Basic controlled form
7.6b-2	Tab switching (Donor / Admin)	`activeTab` state, conditional styling
7.6b-3	Mock authentication logic	Check credentials against hardcoded admin OR localStorage donors
7.6b-4	Call `login()` from AuthContext	`useAuth()` hook, context consumption
7.6b-5	Navigate to home on success	`useNavigate()` from React Router
7.6b-6	Error message for invalid credentials	Error state, `<AlertCircle>` icon

Test it: Login with `admin@raktasetu.com` / `admin123` → should navigate to admin. Register a donor, then login with those credentials.

☑ Phase 7 Checkpoint

- ☐ Contact form validates and shows success/error states
- ☐ Donor registration saves to localStorage
- ☐ Emergency request saves to localStorage
- ☐ Blood bank search filters work correctly
- ☐ Login works for both admin and registered donors
- ☐ User session persists after page refresh (check localStorage)

Phase 8 — Home Page, Animations & Polish (Everyone) ✨

> **Duration:** ~2–3 weeks | **Goal:** The wow factor

8.1 `AnimatedSection.jsx` (Member 2)

Skills needed: Framer Motion basics

What it does: Wraps any content in a fade-in-on-scroll animation

```
Complexity: ★★ (~20 lines)
```

What to learn:

- `import { motion } from 'framer-motion'`
- `<motion.div initial={...} whileInView={...} viewport={...}>` — animate when element enters viewport
- `initial={{ opacity: 0, y: 40 }}` → `whileInView={{ opacity: 1, y: 0 }}`

8.2 `CountUp.jsx` (Member 2)

Skills needed: `useState`, `useEffect`, `useRef`, `IntersectionObserver`

What it does: Animates a number from 0 to `target` when it scrolls into view

```
Complexity: ★★★ (~30 lines)
```

What to learn:

- `IntersectionObserver` API — detects when an element is visible
- Animation loop using `requestAnimationFrame` or `setInterval`
- `useRef` to hold the DOM element reference

8.3 `Home.jsx` + `Home.css` 🟤 *Largest Page (477 lines)*

Skills needed: Everything you've learned so far

What it does: The main landing page with hero section, stats, workflows, features, CTA sections

Break into sections (build one at a time):

Section	Lines	Description	Key Skills
Hero Section	~80	Large banner with headline, subtext, CTA buttons, floating decorative elements	CSS gradients, positioning, responsive design
Stats Bar	~40	Row of animated numbers (donors, blood banks, etc.)	`CountUp` component, flexbox

Section	Lines	Description	Key Skills
User Workflow Cards	~60	"Order Blood", "Schedule Donation", "Emergency" cards	Card grid, icons, `<Link>`
How It Works Steps	~50	3-step visual process	Numbered steps, CSS timeline
Featured Blood Banks	~50	Cards showing top blood banks from data	Import from `bloodBanks.js`, `.slice()`
Features Grid	~50	Grid of `<FeatureCard>` components	Reusing components
CTA Section	~30	Call-to-action banner with buttons	CSS gradients, buttons
Testimonials	~40	Donor/patient quotes	Card layout

> [!TIP] Build each section as a separate commit. Test responsiveness after each section.

8.4 Responsive Polish (Member 2)

Go through every page and every component:

- ☐ Test at 375px (mobile), 768px (tablet), 1024px (laptop), 1440px (desktop)
- ☐ Fix any overflow, text clipping, or layout breaks
- ☐ Ensure touch-friendly tap targets (minimum 44×44px)

8.5 Performance Optimization (Member 1 + Member 4)

Update `vite.config.js`:

- Code splitting: `manualChunks` for vendor, icons
- Minification with `terser`
- Lazy loading pages with `React.lazy()`

Accessibility:

- All images have `alt` text
- All interactive elements have `aria-label`
- Focus styles are visible

8.6 Deployment to Vercel (Member 1)

Steps:

1. Push code to GitHub
2. Go to vercel.com and sign up with GitHub
3. Import the repository
4. Vercel auto-detects Vite, builds, and deploys

5. Get your live URL!

```
# Build locally first to verify
npm run build
npm run preview
```

☑ Phase 8 Checkpoint

- ☐ Home page has all sections with animations
- ☐ Site is responsive on all device sizes
- ☐ All pages lazy-load (check Network tab in DevTools)
- ☐ Site is deployed and accessible via a public URL
- ☐ No console errors in production

📅 Suggested Timeline

Week	Phase	Focus
1–2	Phase 1	HTML + CSS fundamentals (everyone together)
3–4	Phase 2	JavaScript essentials (everyone together)
5–7	Phase 3	React fundamentals (everyone together)
8	Phase 4	Project setup (Member 1 leads)
9–10	Phase 5	Shared components (Member 1 + Member 2)
10–11	Phase 6	Static content pages (Member 5, parallel with Phase 5)
12–15	Phase 7	Interactive pages (Member 3 + Member 4)
16–18	Phase 8	Home page, animations, polish, deploy (everyone)

> **Total estimated time: ~4.5 months** (learning + building, with ample time)

🔄 Git Workflow for the Team

```
graph TD
    A[main branch] --> B[Member creates feature branch]
    B --> C["git checkout -b feature/navbar"]
    C --> D[Write code + test locally]
    D --> E[git add . && git commit]
    E --> F[git push origin feature/navbar]
    F --> G[Create Pull Request on GitHub]
    G --> H[Another member reviews the code]
    H --> I[Merge to main]
```

Branch naming convention:

- `feature/navbar` — new feature
- `fix/login-validation` — bug fix
- `style/home-responsive` — styling only

🧪 How to Test Each Component

What to Test	How to Test
Visual layout	Open in browser, resize window, check mobile view (F12 → toggle device toolbar)
Navigation	Click every link, verify correct page loads
Forms	Submit empty → check errors. Fill correctly → check success. Check localStorage in DevTools
Search/Filters	Try each filter alone and combined. Clear and verify reset
Auth	Register → Login → Refresh page → Still logged in? Logout → Session gone?
Responsive	Chrome DevTools → Device Toolbar → Test at 375px, 768px, 1024px
Errors	Open Console tab for any red errors. Fix them all before moving on

---

🆙 Future Upgrades (After MVP)

Once you've built the full project, here's how to level up:

Upgrade	What to Learn	Difficulty
Real Backend	Node.js + Express + MongoDB	★★★★★
Real Authentication	JWT tokens, bcrypt password hashing	★★★★
Admin Dashboard	Protected routes, CRUD operations	★★★★
Email Notifications	Nodemailer or SendGrid API	★★★
Maps Integration	Google Maps API for blood bank locations	★★★
Real-time Updates	WebSocket for emergency notifications	★★★★
Mobile App	React Native (reuse your React knowledge!)	★★★★
TypeScript	Type safety for fewer runtime bugs	★★★
Testing	Jest + React Testing Library	★★★

> [!IMPORTANT] **The Golden Rule:** Build small, test immediately, commit often. Never write more than 30 lines of code without checking if it works in the browser. If something breaks, `git stash` your changes and debug from the last working state.